ByteMart Digital

THE COMPLETE GUIDE

Swagger & OpenAPI

API Documentation for Spring Boot Developers

A focused, practical guide to documenting APIs with OpenAPI and Swagger — the specification, the tooling, and auto-generating live docs from Spring Boot with springdoc.

7	30+	6	3
Chapters	Examples	Diagrams	Modules

© 2026 ByteMart Digital · All Rights Reserved

ByteMart Digital

Swagger & OpenAPI Documentation

First Edition · 2026

Copyright © 2026 ByteMart Digital. All rights reserved.
No part of this publication may be reproduced, distributed, or transmitted in any form or by any means — including photocopying, recording, screenshotting, or other electronic or mechanical methods — without the prior written permission of the publisher, except for brief quotations used in reviews and certain other non-commercial uses permitted by copyright law.

This product is licensed for personal use by the purchaser only. Sharing, reselling, or redistributing this file in whole or in part is strictly prohibited and constitutes a violation of copyright law.

Disclaimer
The information in this book is provided in good faith and for educational purposes. While every effort has been made to ensure accuracy, the author and publisher assume no responsibility for errors, omissions, or outcomes arising from the use of the code or information contained herein. All code examples are provided “as is”.

Trademarks
Swagger, OpenAPI, Spring, and other product names, logos, and brands referenced are the property of their respective owners and are used for identification purposes only. This guide is an independent educational work.

Published by ByteMart Digital

Web: bytemartdigital.com
Email: contact@bytemartdigital.in
Instagram: instagram.com/bytemartdigital.in
Telegram: t.me/bytemartcodingcommunity

Contents

1. What Is OpenAPI & Swagger? 4
2. The OpenAPI Document 6
3. Schemas & Data Models 9
4. Springdoc: Auto-Generating Docs 12
5. Annotating Your API 15
6. Documenting Security 17
7. Swagger UI & Best Practices 20

MODULE 1 · OPENAPI FUNDAMENTALS

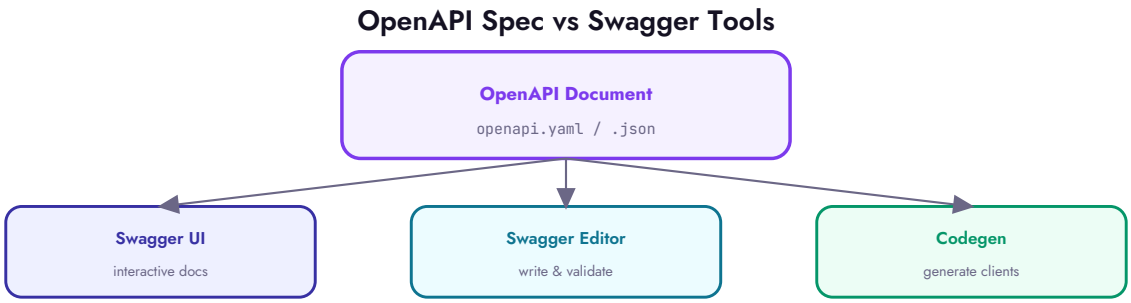
CHAPTER 01

What Is OpenAPI & Swagger?

The specification, the tools that read it, and why a documented API is a better API.

An API that is not documented is hard to use, hard to test, and hard to trust. **OpenAPI** solves this by describing your API in a single, standard, machine-readable document — and **Swagger** is the family of tools that turn that document into interactive docs, clients, and more.

Spec versus tools



OpenAPI is the standard document; Swagger is the family of tools that read it.

The two names cause endless confusion, so be precise: **OpenAPI** is the specification — the format of the document (formerly called the Swagger Specification). **Swagger** is a set of tools — Swagger UI, Swagger Editor, and Codegen — that read and act on an OpenAPI document.

Why document with OpenAPI

- **Interactive docs** — Swagger UI renders a live, try-it-out API explorer.
- **A single source of truth** — one document describes every endpoint.
- **Code generation** — generate client SDKs and server stubs.
- **Contract** — frontend and backend agree on the shape up front.

One document, many outputs

OpenAPI is a standard, machine-readable document describing your API; Swagger is the tooling that reads it (UI, Editor, Codegen). Document once and get interactive docs, generated clients, and a shared contract.

MODULE 1 · OPENAPI FUNDAMENTALS

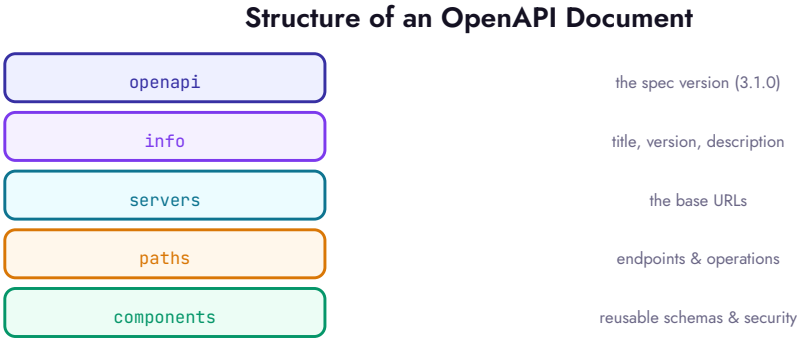
CHAPTER 02

The OpenAPI Document

The top-level structure, and how paths, operations, and responses are described.

An OpenAPI document is a single YAML or JSON file with a small set of top-level keys. The `paths` section is its heart — describing every endpoint, its inputs, and its responses.

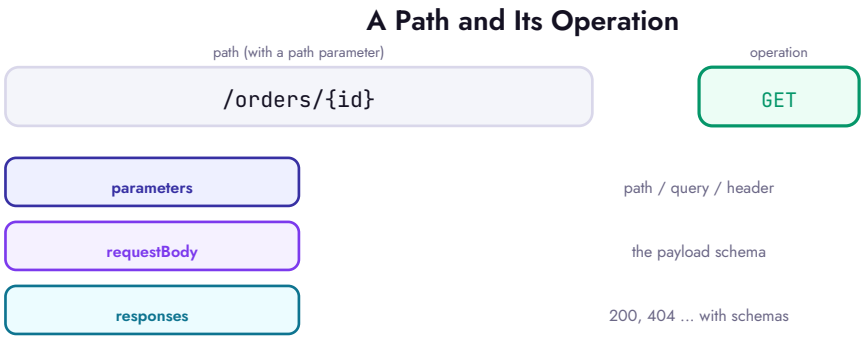
The top-level keys



A handful of top-level keys describe the whole API in one machine-readable document.

Key	Describes
openapi	the spec version (3.1.0)
info	title, version, description
servers	base URLs for the API
paths	the endpoints and operations
components	reusable schemas, responses, security

Paths and operations



Each path holds operations by method; each operation has parameters, a body, and responses.

Each path holds one or more **operations** (by HTTP method). An operation describes its `parameters` (path, query, header), its `requestBody`, and its `responses` — keyed by status code, each with a schema.

Document error responses, not just the happy path.

yaml

```yaml
openapi: 3.1.0
info: { title: Orders API, version: 1.0.0 }
servers: [{ url: https://api.bytemart.com }]
paths:
  /orders/{id}:
    get:
      summary: Get an order
      parameters:
        - { name: id, in: path, required: true, schema: { type: integer } }
      responses:
        '200':
          description: The order
          content:
            application/json:
              schema: { $ref: '#/components/schemas/Order' }
        '404': { description: Not found }
```

A few keys, paths at the heart

An OpenAPI document uses openapi, info, servers, paths, and components. Each path holds operations by method, each with parameters, a request body, and responses by status code — document errors too.

MODULE 1 · OPENAPI FUNDAMENTALS

CHAPTER 03

Schemas & Data Models

Defining your data shapes once and reusing them everywhere with $ref.

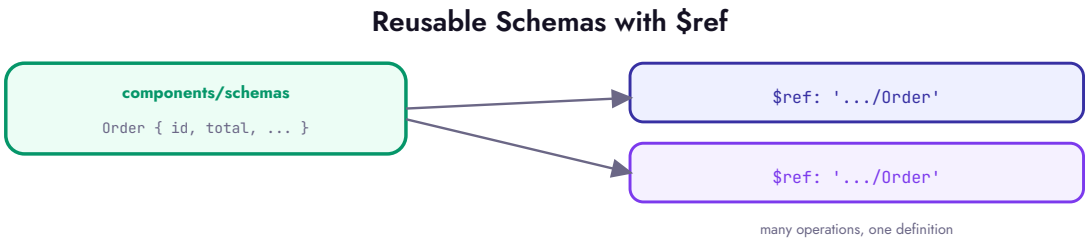
APIs exchange structured data, and OpenAPI describes that data with **schemas**. Defining each model once under `components` and referencing it with `$ref` keeps your spec DRY and consistent.

Defining a schema

yaml

```yaml
components:
  schemas:
    Order:
      type: object
      required: [id, total]
      properties:
        id:     { type: integer, example: 42 }
        total:  { type: number, format: float }
        status: { type: string, enum: [pending, paid, shipped] }
        items:
          type: array
          items: { $ref: '#/components/schemas/Item' }
```

Reuse with $ref



Define a model once under components, then reference it with $ref everywhere it is used.

A `$ref` points to a schema defined elsewhere — `'#/components/schemas/Order'`. Instead of repeating the `Order` shape in every operation, you reference it, so a change in one place updates every use.

Keyword	Specifies
type / format	integer, string, number (date, email)
required	which properties must be present
enum	an allowed set of values
example	a sample value shown in docs

Model once, reference everywhere

Schemas describe data shapes under components, with type, format, required, enum, and example. Reference them with $ref so each model is defined once and reused — no duplication, always consistent.

MODULE 2 · OPENAPI IN SPRING BOOT

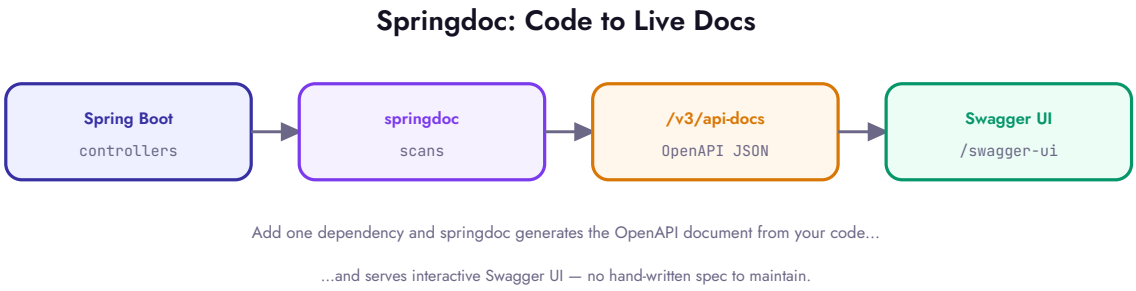
CHAPTER 04

Springdoc: Auto-Generating Docs

Producing an OpenAPI document and Swagger UI straight from your Spring Boot code.

Hand-writing an OpenAPI document is tedious and drifts out of sync with the code. For Spring Boot, **springdoc-openapi** generates the document automatically from your controllers — and serves interactive Swagger UI for free.

From code to live docs



Add one dependency and springdoc generates the OpenAPI document and serves Swagger UI.

For Spring Boot 3, add the springdoc starter. It scans your controllers, models, and annotations at startup and produces an OpenAPI 3 document — no configuration required to get started.

```xml
<dependency>
  <groupId>org.springdoc</groupId>
  <artifactId>springdoc-openapi-starter-webmvc-ui</artifactId>
  <version>2.x.x</version>
</dependency>
```

The two endpoints

URL	Serves
/v3/api-docs	the OpenAPI document (JSON)
/swagger-ui.html	interactive Swagger UI

WATCH OUT

The older **Springfox** library is unmaintained and does not support recent Spring Boot versions. Use **springdoc-openapi** for any modern Spring Boot 3 project — do not follow old tutorials that use Springfox.

Docs generated from code

springdoc-openapi generates an OpenAPI 3 document and serves Swagger UI from your Spring Boot controllers with one dependency — exposing /v3/api-docs and /swagger-ui.html. Use it, not the unmaintained Springfox.

MODULE 2 · OPENAPI IN SPRING BOOT

CHAPTER 05

Annotating Your API

Enriching the generated docs with descriptions, examples, and response details.

Springdoc infers a lot from your code, but annotations make the docs genuinely useful — adding summaries, descriptions, examples, and precise response definitions that code alone cannot convey.

Describing operations and models

java

```java
@Operation(summary = "Get an order",
           description = "Returns a single order by its id")
@ApiResponses({
  @ApiResponse(responseCode = "200", description = "Found"),
  @ApiResponse(responseCode = "404", description = "Not found")
})
@GetMapping("/orders/{id}")
public Order get(@PathVariable Long id) { ... }

public record Order(
    @Schema(description = "Unique id", example = "42") Long id,
    @Schema(description = "Order status", example = "paid") String status
) {}
```

The key annotations

Annotation	Adds
@Operation	summary and description
@ApiResponse	a documented response
@Parameter	parameter description and example
@Schema	model field docs and constraints
@Tag	grouping operations in the UI

Annotate methods with `@Operation` and `@ApiResponse`, model fields with `@Schema`, and group related endpoints under a heading with `@Tag`. These flow straight into Swagger UI and the generated document.

Annotations make docs useful

@Operation and @ApiResponse describe endpoints and responses; @Schema documents model fields with examples; @Tag groups operations. These turn auto-generated docs into genuinely helpful ones.

© 2026 ByteMart Digital

MODULE 2 · OPENAPI IN SPRING BOOT

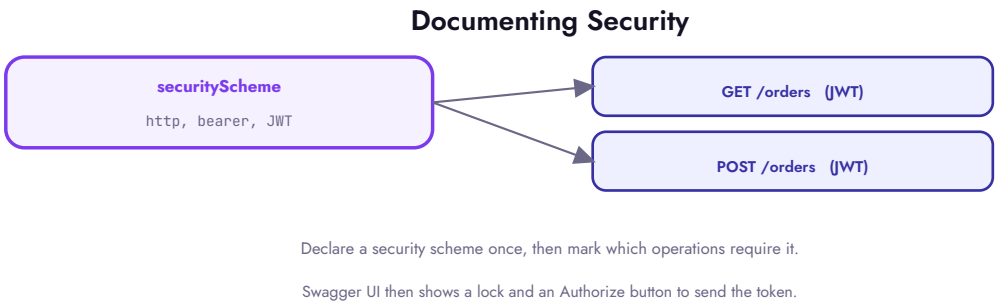
CHAPTER 06

Documenting Security

Describing how your API is authenticated so Swagger UI can send credentials.

Most APIs are protected, and the docs must say how. OpenAPI describes authentication with **security schemes**, and once declared, Swagger UI adds an **Authorize** button so users can try secured endpoints.

Declaring a scheme



Declare a security scheme once, then mark which operations require it.

A security scheme goes under `components`. A JWT bearer API declares an `http` scheme with the `bearer` type and a `JWT` format; operations then reference it via `security`.

```yaml
components:
  securitySchemes:
    bearerAuth:
      type: http
      scheme: bearer
      bearerFormat: JWT
security:
  - bearerAuth: []
```

In Spring Boot

```java
@SecurityScheme(name = "bearerAuth", type = SecuritySchemeType.HTTP,
                scheme = "bearer", bearerFormat = "JWT")
@OpenAPIDefinition(
    security = @SecurityRequirement(name = "bearerAuth"))
public class OpenApiConfig {}
```

Once a scheme is declared, Swagger UI shows an **Authorize** button. A user pastes their token once, and every subsequent try-it-out request includes it — making the docs a working test client for protected endpoints.

Declare auth so docs can use it

Security schemes under components describe how the API authenticates (e.g. bearer JWT); operations reference them. Springdoc generates this from annotations, and Swagger UI adds an Authorize button to send the token.

© 2026 ByteMart Digital

MODULE 3 · SWAGGER UI & PRACTICE

CHAPTER 07

Swagger UI & Best Practices

Customizing the interactive docs, and the habits that keep documentation honest.

Swagger UI is what most people mean by 'Swagger' — the interactive page that renders your OpenAPI document into browsable, testable docs. A few settings shape it, and a few habits keep it trustworthy.

Customizing Swagger UI

Springdoc exposes many settings as properties — change the UI path, sort operations and tags, and toggle try-it-out. An `OpenAPI` bean sets the title, version, and description shown at the top.

properties

```
springdoc.swagger-ui.path=/docs
springdoc.swagger-ui.operationsSorter=alpha
springdoc.swagger-ui.tagsSorter=alpha
springdoc.swagger-ui.tryItOutEnabled=true
```

Design-first vs code-first

Design-first writes the OpenAPI spec before coding — great for teams building client and server in parallel. **Code-first** (springdoc's default) annotates the code and generates the spec — simpler for a single team and it keeps docs close to reality. Both are valid; choose per project.

WATCH OUT

The worst documentation is documentation that lies. If you hand-write specs separately from code, they drift. Generating from code keeps docs honest — they change when the code changes.

Best practices

- **Document every operation and error** — not just the happy path.
- **Add examples** to schemas and parameters — they make docs concrete.
- **Reuse with $ref** — define each model once.
- **Version your API** and reflect it in `info.version`.
- **Secure or disable the docs endpoint** in production.

Shaped to fit, kept honest

Swagger UI renders your spec as browsable, testable docs; springdoc properties and an OpenAPI bean customize it. Choose design-first or code-first, but keep docs in sync — document every operation and error, add examples, and secure the endpoint in production.

© 2026 ByteMart Digital